

ROADGUARD

AI/ML-Based Road Condition Detection System

A Project Report

Submitted in partial fulfillment of the requirements for the
award of the
Diploma in Artificial Intelligence and Machine Learning.

Submitted by:

Atharv shetty— 8080110

Under the guidance of:

K.S SHIRUR



Government Tool Room and Training Centre (GTTC), Belagavi
[Academic Year :2026]

Certificate

This is to certify that the project entitled “ROADGUARD — AI/ML-Based Road Condition Detection System” has been carried out by Atharv Shetty (Roll No. 8080110) in partial fulfillment of the requirements for the Diploma in Artificial Intelligence and Machine Learning during the academic year 2026, under my supervision.

Guide's Signature: _____

Name: K.S Shirur

Head of Department: _____

Name: [HOD's Name]

Declaration

I hereby declare that the project entitled “ROADGUARD — AI/ML-Based Road Condition Detection System” is an original piece of work carried out by me, under the guidance of Miss K.S Shirur, and has not been submitted elsewhere for any other degree or diploma. All datasets, libraries, and external resources used have been duly acknowledged.

Signature: _____

Name: Atharv shetty

Date: _____

Atharv Shetty

Acknowledgement

I would like to express my sincere gratitude to my project guide Miss K.S Shirur, for their valuable guidance, constant encouragement, and support throughout the development of this project. I am also thankful to the Head of Department and all faculty members of Diploma in Artificial Intelligence and Machine Learning for providing the resources and environment necessary to complete this work.

I would also like to acknowledge the creators of the Kaggle “Road Issues Detection Dataset”, whose publicly shared data made the practical implementation of this project possible.

Abstract

Checking roads by hand for damage like potholes and cracks takes a lot of time and is hard to do consistently across a large area. This project, RoadGuard, is a simple AI/ML system that looks at a photo of a road and decides if it is NORMAL or DAMAGED. The whole model is built from scratch using NumPy — no TensorFlow, no PyTorch, no pre-trained models.

The project covers the full pipeline: preparing images with OpenCV, a hand-built perceptron and a small 2-layer neural network, forward and backward propagation written by hand, two loss functions (Mean Squared Error and Binary Cross-Entropy), two optimizers (SGD and Adam), and evaluation metrics calculated manually (Accuracy, Precision, Recall, F1 Score, Confusion Matrix). A Flask API serves predictions to a React website that supports both photo upload and a live webcam.

The model was trained on a subset of the Kaggle “Road Issues Detection Dataset”, plus extra normal-road photos added separately, since the original dataset has no “normal road” category at all. On 1,418 test images it had never seen, the model reached 90.34% accuracy, 95.43% precision, 93.14% recall, and an F1 score of 94.27% — showing that even a fully hand-built model can do a solid job on a real classification task, while staying simple enough to fully explain.

Table of Contents

Certificate.....	2
Declaration	3
Acknowledgement	4
Abstract.....	5
Table of Contents.....	6
Chapter 1: Introduction.....	8
1.1 Background	8
1.2 Problem Statement.....	8
1.3 Objectives.....	8
1.4 Scope of the Project.....	9
1.5 Organization of the Report.....	9
Chapter 2: Literature Review / Existing Approaches.....	10
Chapter 3: System Requirements	11
3.1 Functional Requirements	11
3.2 Non-Functional Requirements.....	11
3.3 Hardware Requirements	11
3.4 Software Requirements	12
Chapter 4: System Design.....	13
4.1 System Architecture.....	13
4.2 Data Flow Diagram	14
4.3 Module Description	15
Chapter 5: Dataset	16
5.1 Dataset Source	16
5.2 Class Construction.....	16
5.3 Dataset Split	16
5.4 Class Imbalance	16
Chapter 6: Methodology and Implementation	17
6.1 Image Preprocessing	17
6.2 NumPy Fundamentals (Practical Topic 1).....	17
6.3 Perceptron and Activation Functions (Practical Topic 2).....	17
6.4 Neural Network Architecture (Practical Topic 3)	18
6.5 Loss Functions and Optimizers (Practical Topic 4)	18
6.6 Training Procedure.....	19

Chapter 7: Backend API and Frontend Application	20
7.1 Flask REST API.....	20
7.2 React Frontend	20
Chapter 8: Screenshots	21
8.1 Dashboard	21
8.2 Detect Road Damage — Normal Result	21
8.3 Detect Road Damage — Damaged Result	22
8.4 Prediction History	22
8.5 Model Performance	23
Chapter 9: Testing.....	24
Chapter 10: Results and Discussion	25
10.1 Training Behavior.....	25
10.2 Optimizer Comparison: SGD vs. Adam	25
10.3 Loss Function Comparison: MSE vs. Binary Cross-Entropy	25
10.4 Final Model Evaluation.....	25
Confusion Matrix	26
10.5 Discussion of Class Imbalance.....	27
Chapter 11: Conclusion and Future Scope.....	28
11.1 Conclusion	28
11.2 Limitations.....	28
11.3 Future Scope.....	28
References.....	29
Appendix A: Project Structure	30

Chapter 1: Introduction

1.1 Background

Roads matter a lot — their condition affects safety, travel time, and how much we spend fixing vehicles. Damage like potholes and cracks usually starts small and gets worse if nobody notices it. Right now, checking road condition is mostly done by people manually surveying roads, which is slow and gives different results depending on who is checking.

Image classification can help automate part of this job using ordinary photos. RoadGuard explores this at a learning level: instead of using a ready-made deep-learning library, every step — from basic matrix math to backpropagation — is coded by hand in NumPy, so nothing about how the model works is hidden.

1.2 Problem Statement

Given a photo of a road (from a file or a webcam), decide automatically whether the road is NORMAL or DAMAGED. The model must be built without deep-learning frameworks, and every result shown must come from real data — nothing fabricated.

1.3 Objectives

- Build an image classifier using a hand-written NumPy neural network (no CNN, no external ML framework).
- Cover four core practical topics — NumPy basics, perceptron and activation functions, forward/backpropagation, and loss functions/optimizers — in one working system.
- Build a genuine two-class (Normal / Damaged) dataset from a multi-category public dataset, without inventing labels.
- Serve predictions through a REST API (Flask) and a browser app (React) that supports both file upload and live camera capture.
- Only show real, measured performance numbers, calculated by hand without third-party evaluation libraries.

1.4 Scope of the Project

This is a diploma-level learning project, not a production inspection tool. It looks at one photo at a time and says Normal or Damaged — it does not point out exactly where the damage is, how bad it is, or work on video. It is built to run on an ordinary student laptop with no GPU needed.

1.5 Organization of the Report

Chapter 2 looks at existing approaches. Chapter 3 covers requirements. Chapter 4 shows the system design. Chapter 5 explains the dataset. Chapter 6 covers how each ML piece was built. Chapter 7 covers the Flask API and React app. Chapter 8 shows screenshots of the working app. Chapter 9 covers testing. Chapter 10 covers results. Chapter 11 wraps up and lists future improvements

Chapter 2: Literature Review / Existing Approaches

People have tried different ways to spot road damage automatically. Older methods use basic image processing, like edge detection, to find cracks. Newer methods use deep learning — CNNs and object-detection tools like YOLO — which can both classify and point out exactly where a pothole is in the photo.

These deep-learning methods usually score higher, but they need large labeled datasets, a GPU, and ready-made libraries that hide how the math actually works. Since the goal of this diploma project is to show the basic building blocks — matrix math, perceptrons, backpropagation, loss functions, and optimizers — those libraries are avoided on purpose in favor of a fully hand-built NumPy version.

RoadGuard sits in between these two extremes: it uses a real public dataset and a genuine (if small) neural network, while keeping every calculation visible and explainable, choosing clarity over top-tier accuracy.

Chapter 3: System Requirements

3.1 Functional Requirements

- The system shall accept a road image via file upload (JPG/PNG) or webcam capture.
- The system shall preprocess the image using a fixed OpenCV pipeline (resize, color conversion, normalization).
- The system shall classify the image as NORMAL ROAD or DAMAGED ROAD with an associated probability.
- The system shall display prediction history to the user, stored locally in the browser.
- The system shall display real evaluation metrics (accuracy, precision, recall, F1, confusion matrix) once the model has been trained.

3.2 Non-Functional Requirements

- The system shall run on a standard laptop without requiring a GPU.
- Prediction latency for a single image shall be low enough for interactive use (well under one second on typical hardware).
- No component of the system shall fabricate results; if the model is untrained, the system shall state this explicitly rather than showing placeholder values.

3.3 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / equivalent or higher
RAM	4 GB (8 GB recommended)
Storage	2 GB free space (for dataset and dependencies)
Camera	Any USB or built-in webcam (for the Camera feature)

3.4 Software Requirements

Component	Technology / Version
Backend language	Python 3.10+
ML libraries	NumPy, OpenCV (opencv-python), Matplotlib, Pandas
Web framework (backend)	Flask, Flask-CORS
Frontend framework	React 18 (via Vite)
Package managers	pip, npm
Dataset source	Kaggle — Road Issues Detection Dataset

Chapter 4: System Design

4.1 System Architecture

RoadGuard has three simple layers: a React app the user sees, a Flask API in the middle, and a NumPy model that does the actual thinking.

RoadGuard System Architecture

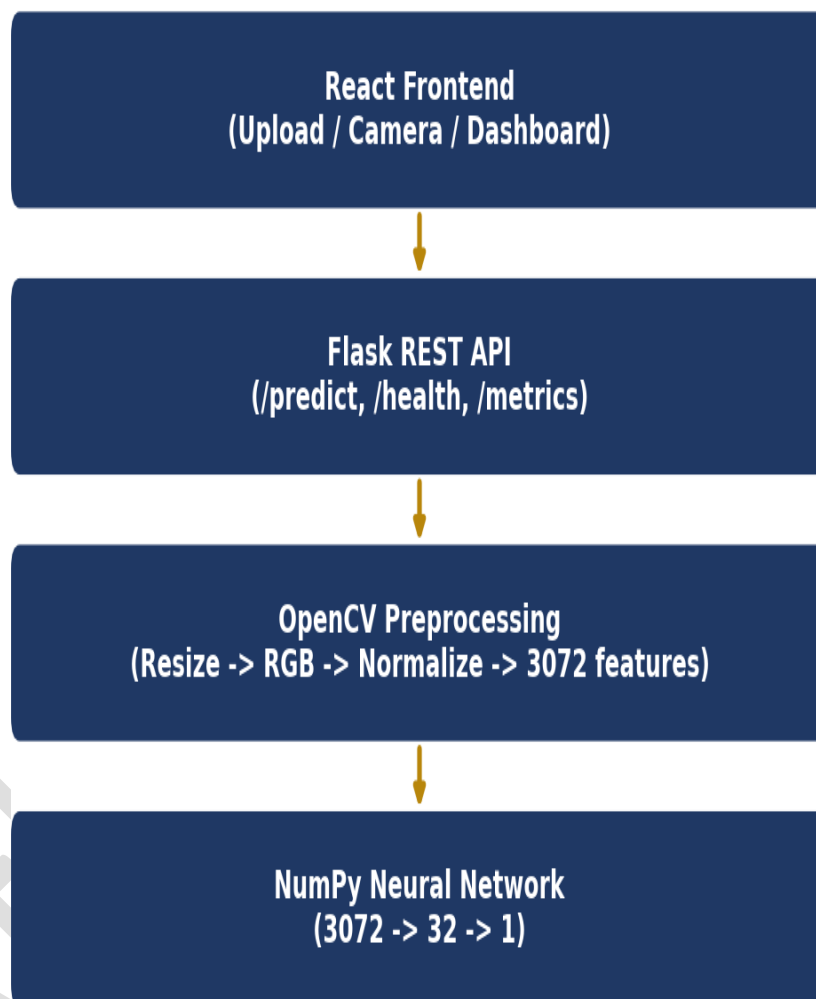


Figure 4.1 — System architecture: each layer calls the one below it.

4.2 Data Flow Diagram

This diagram shows the full journey of an image, from the raw dataset all the way to a prediction shown on screen:

End-to-End Data Flow

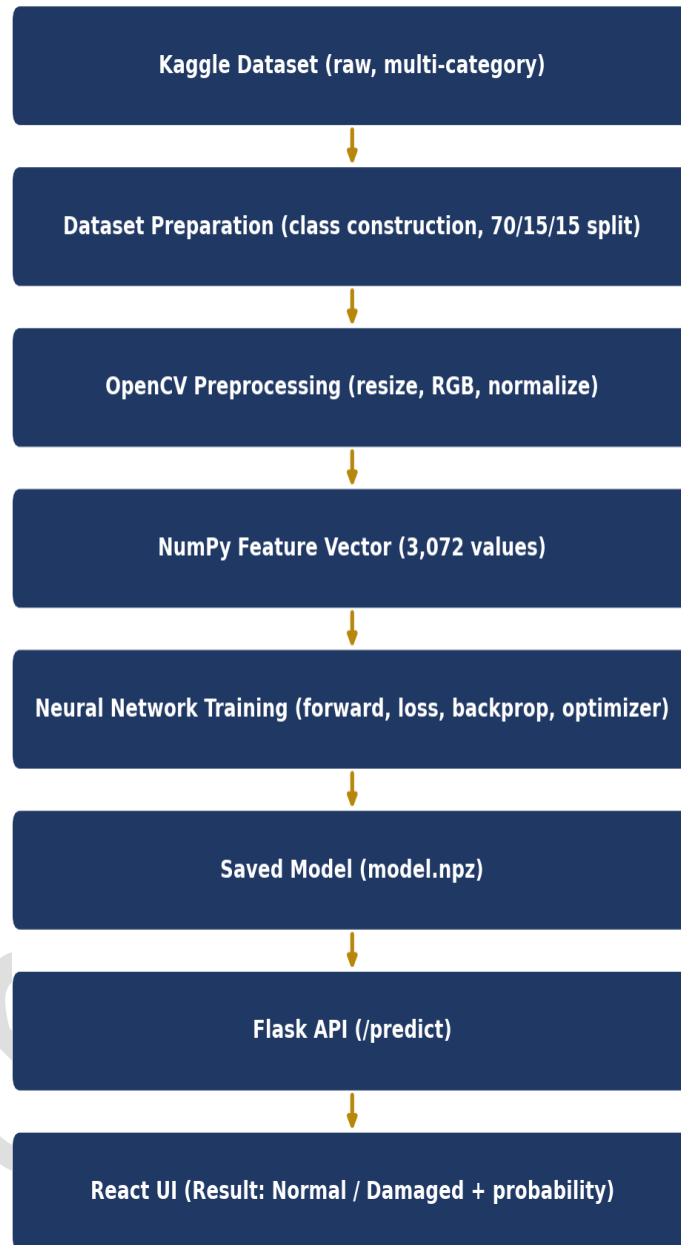


Figure 4.2 — End-to-end data flow, from dataset to prediction.

4.3 Module Description

Module (File)	Responsibility
practical_numpy.py	Standalone demonstration of NumPy array operations and the dot-product computation underlying every dense layer.
perceptron.py	From-scratch single-layer perceptron with a manual weight-update rule.
activation_visualization.py	Manual implementations of Sigmoid, Tanh, and ReLU with plotted outputs.
preprocessing.py	The single OpenCV pipeline used identically for training, validation, testing, and live prediction.
neural_network.py	The 2-layer neural network: forward propagation, backpropagation, loss functions, and optimizers (SGD, Adam).
data_loader.py	Memory-efficient mini-batch loading of images directly from disk.
prepare_dataset.py	Recursively scans the raw Kaggle dataset, constructs the Normal/Damaged classes, and performs a reproducible 70/15/15 split.
train.py	Orchestrates training, saves the model, and generates comparison charts (optimizer and loss-function comparisons).
evaluation.py	Computes Accuracy, Precision, Recall, F1, and the Confusion Matrix manually, and generates the confusion-matrix chart.
predict.py	Loads the trained model and runs inference on a single image.
app.py	Flask REST API exposing /health, /predict, and /metrics.
src/ (React components)	Dashboard, Detect, Camera, History, Performance, and How-It-Works pages.

Chapter 5: Dataset

5.1 Dataset Source

The primary dataset used is the publicly available Kaggle “Road Issues Detection Dataset” (<https://www.kaggle.com/datasets/programmerdai/road-issues-detection-dataset>), containing approximately 9,660 high-resolution RGB images across several road- and public-infrastructure-issue categories, including Broken Road Sign Issues, Damaged Road Issues, Pothole Issues, Illegal Parking Issues, Mixed Issues, and Littering/Garbage Issues.

5.2 Class Construction

Because this project requires only a binary classification (Normal vs. Damaged road), the multi-category dataset was deliberately narrowed rather than used as-is:

- **DAMAGED ROAD:** built only from the “Damaged Road Issues” and “Pothole Issues” categories, which represent genuine road-surface damage.
- Categories unrelated to road-surface condition — Broken Road Sign Issues, Illegal Parking Issues, Mixed Issues, and public cleanliness/litter categories — were explicitly excluded from the Damaged class.
- **NORMAL ROAD:** the source dataset does not contain a dedicated “normal / undamaged road” category, since it is built around road issues. Normal-road images were therefore supplied separately by the project author and merged in via a manual-images folder, rather than being fabricated or inferred from unrelated categories.

This design decision reflects a deliberate constraint of the project: labels are never invented, and if a class cannot be genuinely populated, the dataset-preparation script halts and reports this rather than silently mislabeling data.

5.3 Dataset Split

The combined dataset was split into training, validation, and test sets using a fixed random seed (42) for reproducibility, in a 70% / 15% / 15% ratio. The final split sizes used for training and evaluation in this project were:

Split	Sample Count
Training	6,596
Validation	1,411
Test	1,418

5.4 Class Imbalance

Damaged-road images outnumber normal-road images, causing class imbalance. The preparation script detects this imbalance, which is discussed in Chapter 9 (Results) and Chapter 10 (Future Scope).

Chapter 6: Methodology and Implementation

6.1 Image Preprocessing

A single, consistent preprocessing pipeline is used identically during training, validation, testing, and live prediction, ensuring that the model always sees data in the same form it was trained on:

- Read the image using OpenCV.
- Convert color space from BGR to RGB.
- Resize the image to a fixed 32×32 resolution.
- Normalize pixel values from the $[0, 255]$ range to $[0, 1]$.
- Flatten the $32 \times 32 \times 3$ array into a 3,072-element feature vector.

Additional OpenCV operations — grayscale conversion, Gaussian blur, Canny edge detection, and histogram equalization — were also implemented for demonstration purposes but were deliberately not included in the production pipeline, to keep training, validation, and inference perfectly consistent.

6.2 NumPy Fundamentals (Practical Topic 1)

Before building the neural network, foundational NumPy operations were demonstrated independently: array creation, reshaping, slicing, element-wise matrix operations, and the dot-product-plus-bias computation ($z = X \cdot W + b$) that forms the core arithmetic of every dense layer in the network.

6.3 Perceptron and Activation Functions (Practical Topic 2)

A single-layer perceptron was implemented from scratch, including manually coded weights, bias, weighted-sum computation, prediction, error calculation, and the classic perceptron weight-update rule — without using any pre-built perceptron implementation.

Three activation functions were also implemented manually and visualized from real computed values:

Activation	Formula	Role in RoadGuard
Sigmoid	$1 / (1 + e^{-z})$	Output layer — converts the final score into a probability between 0 and 1.
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	Implemented for demonstration/comparison purposes.
ReLU	$\max(0, z)$	Hidden layer — introduces non-linearity while remaining computationally simple.

6.4 Neural Network Architecture (Practical Topic 3)

The RoadGuard model is a small, fully connected neural network with just two layers:

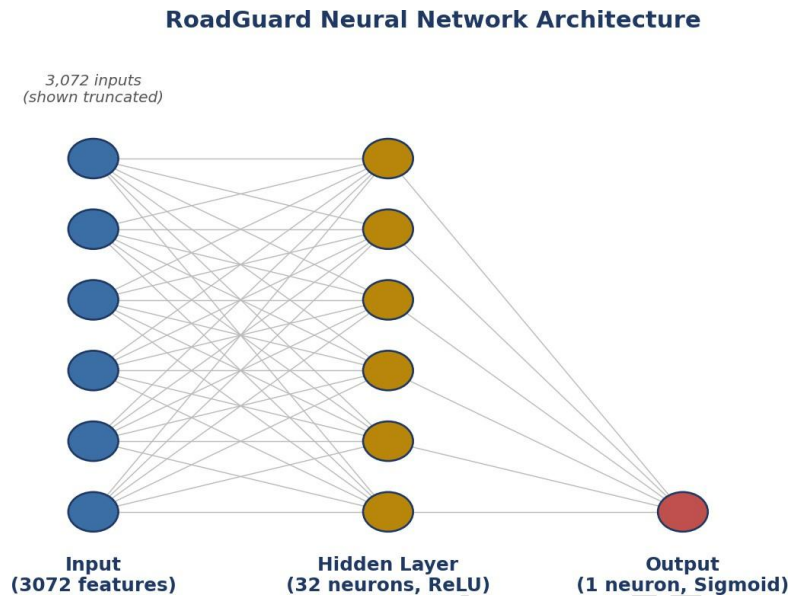


Figure 6.1 — The network: 3,072 inputs, 32 hidden neurons, 1 output.

Forward propagation (making a prediction) works like this: $Z1 = X \cdot W1 + b1$, then $A1 = \text{ReLU}(Z1)$, then $Z2 = A1 \cdot W2 + b2$, then $A2 = \text{Sigmoid}(Z2)$. That final value, $A2$, is the predicted probability that the road is damaged.

Backpropagation (learning from mistakes) was worked out and coded by hand using the chain rule. Starting from the error at the output, gradients are calculated for $W2$, $b2$, $W1$, and $b1$, and used to nudge every weight in the direction that reduces the error.

6.5 Loss Functions and Optimizers (Practical Topic 4)

Two loss functions were implemented manually:

- Mean Squared Error (MSE) — used for comparison purposes.
- Binary Cross-Entropy (BCE) — used as the primary training loss, since it produces cleaner gradients for a sigmoid-activated binary classifier and penalizes confident, incorrect predictions more strongly.

Two optimizers were implemented manually:

- Stochastic Gradient Descent (SGD) — updates weights using a fixed learning rate applied directly to the computed gradient.
- Adam — maintains per-parameter adaptive learning rates using running estimates of the first and second moments of the gradients, generally converging faster than plain SGD.

No framework-provided optimizer (such as Keras' or PyTorch's built-in Adam/SGD) was used; both were implemented directly with NumPy array operations.

6.6 Training Procedure

Training uses mini-batch gradient descent: images are loaded from disk in small batches (rather than loading the entire dataset into memory at once), preprocessed, passed through the network, scored against Binary Cross-Entropy loss, and used to update the network's weights via the selected optimizer. This process repeats for a fixed number of epochs, with both training and validation loss tracked and plotted after each epoch to monitor learning progress and detect overfitting.

Atharv Shetty

Chapter 7: Backend API and Frontend Application

7.1 Flask REST API

The trained model is served through a lightweight Flask application exposing three endpoints:

Endpoint	Method	Description
/health	GET	Returns {"status": "ok"} to confirm the API is running.
/predict	POST	Accepts a multipart form image, runs preprocessing and inference, and returns the predicted label and probability.
/metrics	GET	Returns the real evaluation metrics computed by evaluation.py, or an explicit “not trained” message if no model/metrics exist yet.

Cross-Origin Resource Sharing (CORS) is enabled so that the React frontend, served from a different local port, can communicate with the API without browser restrictions.

7.2 React Frontend

The frontend is a single-page React application (built with Vite) organized into six sections:

- Dashboard — project overview and quick navigation.
- Detect Road Damage — image upload with drag-and-drop, preview, and prediction.
- Camera — live webcam capture using the browser's navigator.mediaDevices API, sending the frame for prediction only when the user explicitly presses Predict.
- Prediction History — a locally stored (localStorage) log of past predictions with date, time, label, and probability.
- Model Performance — displays real accuracy, precision, recall, F1 score, and generated charts; shows an explicit message if the model has not yet been trained.
- How It Works — an explanatory data-flow diagram and a mapping of each practical topic to its implementing file, intended to support viva explanation.

Chapter 8: Screenshots

This chapter shows what the finished application looks like and how it behaves.

8.1 Dashboard

The landing page. It gives a quick summary of the model and two buttons to jump straight into using it.

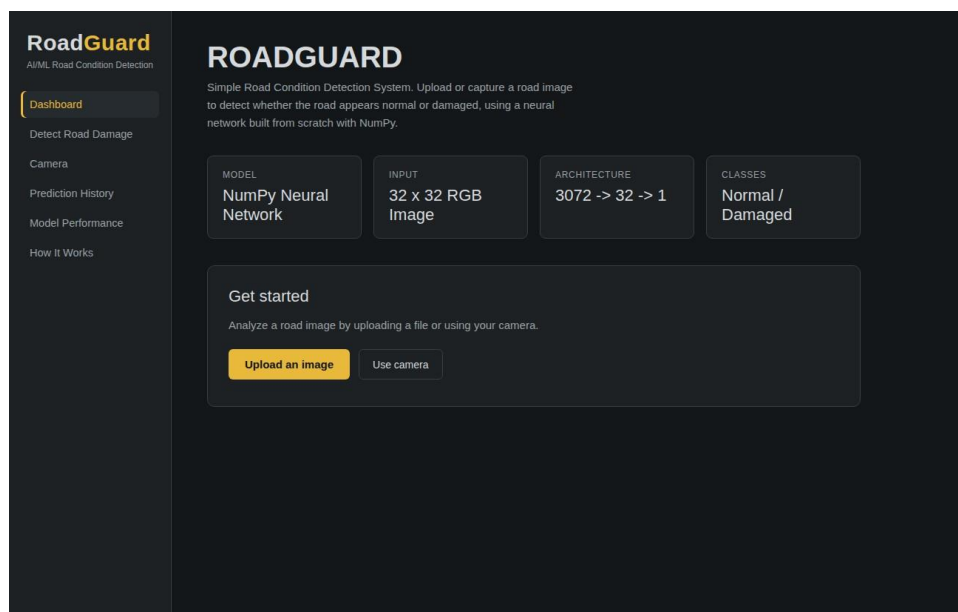


Figure 8.1 — Dashboard page.

8.2 Detect Road Damage — Normal Result

After uploading a photo of a normal road and pressing Detect, the app shows a green result with the predicted probability.

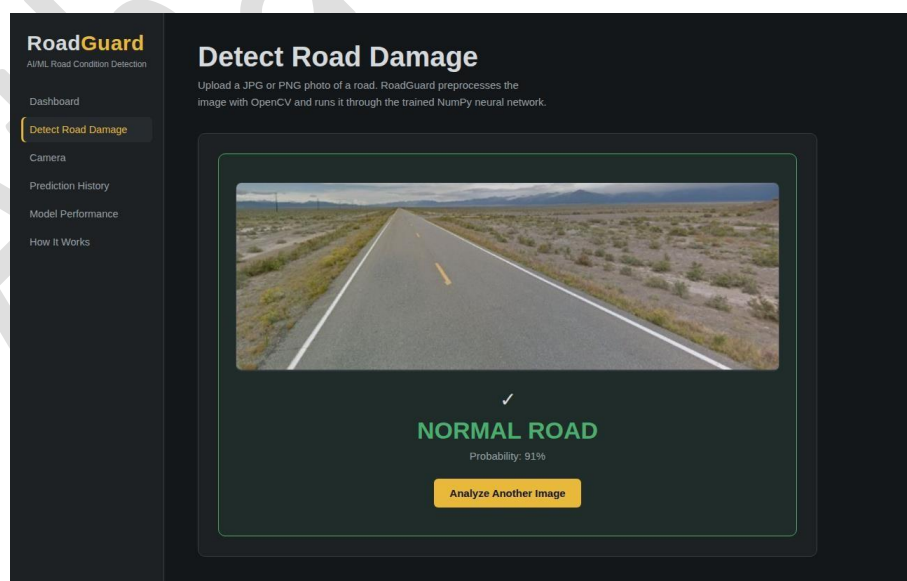


Figure 8.2 — A normal-road prediction.

8.3 Detect Road Damage — Damaged Result

For a damaged road, the same screen turns red and shows a warning icon instead. (The image below is a stylized example used to illustrate the layout, not an actual dataset photo.)

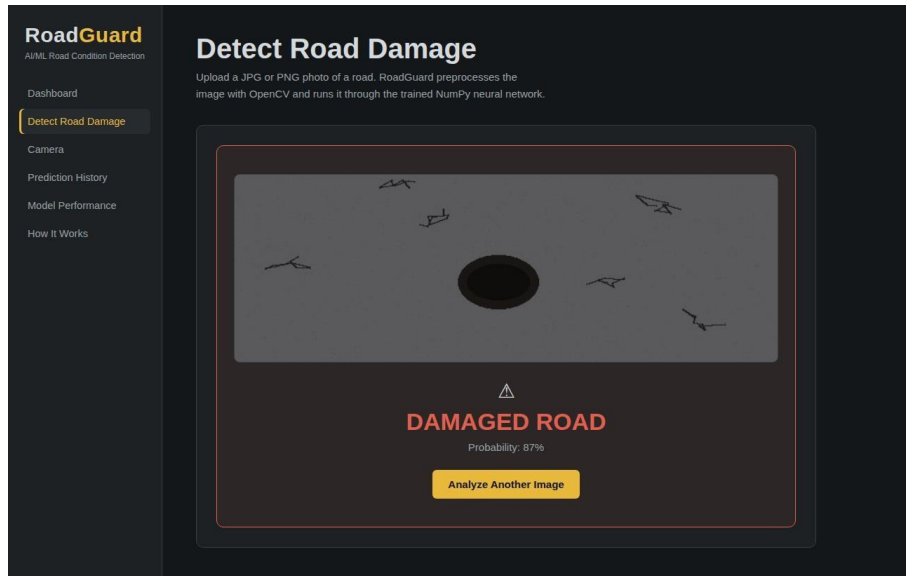


Figure 8.3 — A damaged-road prediction.

8.4 Prediction History

Every prediction the user makes is saved in the browser and listed here, newest first.

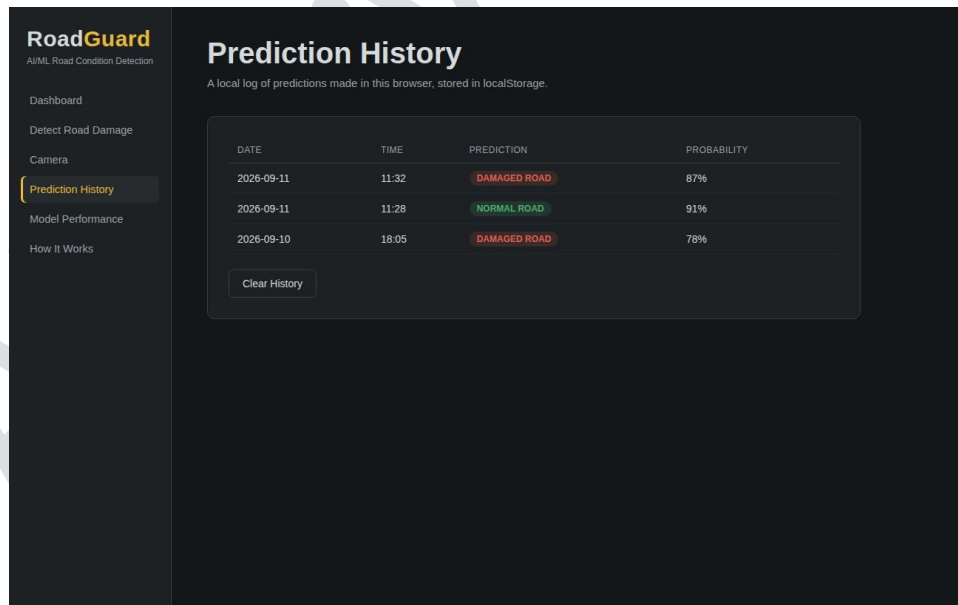


Figure 8.4 — Prediction history page.

8.5 Model Performance

This page shows the model's real, measured accuracy, precision, recall, and F1 score — pulled live from the Flask API, never hard-coded.

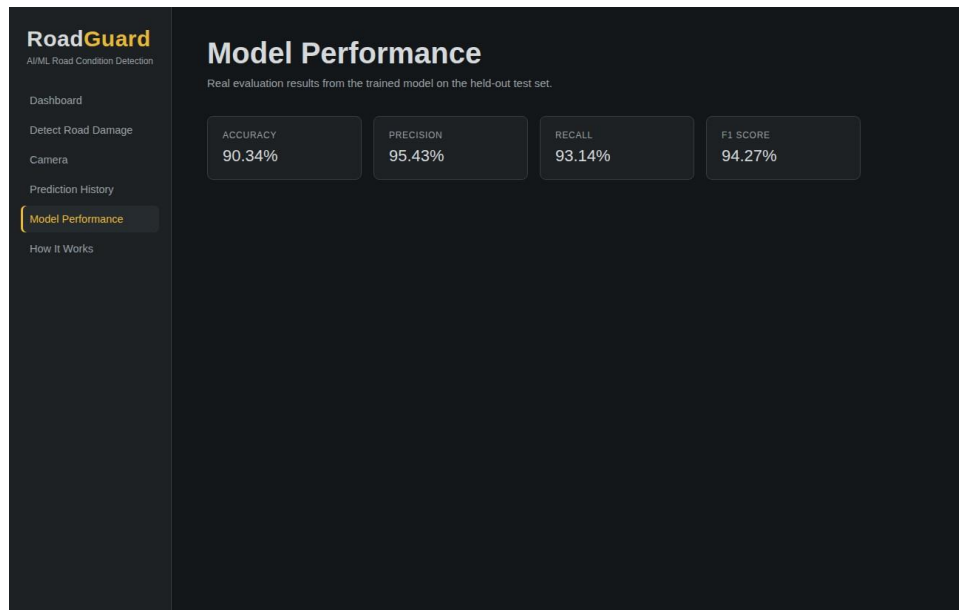


Figure 8.5 — Model performance page, showing the real metrics from Chapter 10.

Chapter 9: Testing

The system was tested at both the component (unit) level and as an integrated end-to-end system, in the following sequence:

#	Test	Expected Outcome	Result
1	Python syntax check on all backend modules	All files compile without error	Pass
2	practical_numpy.py execution	NumPy operations run and matrix visualization is generated	Pass
3	perceptron.py execution	Perceptron converges on a toy linearly separable dataset	Pass
4	activation_visualization.py execution	Sigmoid, Tanh, ReLU plots generated from real computed values	Pass
5	prepare_dataset.py on raw Kaggle data	Correct categories identified; class counts and imbalance reported	Pass
6	train.py full run	Model trained for 30 epochs; loss decreases; model.npz saved	Pass
7	Optimizer comparison (SGD vs Adam)	Adam converges faster than SGD on the same data	Pass
8	Loss comparison (MSE vs BCE)	BCE continues improving; MSE plateaus early (vanishing gradient with sigmoid)	Pass
9	evaluation.py on test set	Real Accuracy/Precision/Recall/F1 and confusion matrix computed	Pass
10	Flask /health endpoint	Returns {"status": "ok"}	Pass
11	Flask /predict with a real image	Returns correct JSON with prediction and probability	Pass
12	Flask /metrics endpoint	Returns real computed metrics	Pass
13	React image upload flow	Preview displayed; prediction and probability shown after Detect	Pass
14	React webcam capture flow	Camera starts, captures on demand, predicts only after explicit user action	Pass
15	Prediction history (localStorage)	Entries persist across page reloads; Clear History empties the log	Pass
16	Model Performance page before training	Displays "Train the model first..." message, no fabricated numbers	Pass

Chapter 10: Results and Discussion

10.1 Training Behavior

The main model was trained for 30 epochs using Adam and Binary Cross-Entropy loss. Training loss dropped steadily from 0.382 at the start to 0.168 by the end. Validation loss also dropped, from 0.306 to about 0.202, and stayed close to the training loss the whole time — a good sign that the model actually learned useful patterns instead of just memorizing the training photos.

10.2 Optimizer Comparison: SGD vs. Adam

Trained under the exact same conditions for 10 epochs, Adam reached a loss of 0.222, while plain SGD only got to 0.331. This matches what theory predicts — Adam adjusts its learning rate per parameter, so it converges faster than SGD's fixed step size — and here it's backed up by an actual training run, not just assumed.

10.3 Loss Function Comparison: MSE vs. Binary Cross-Entropy

When paired with a sigmoid output, MSE loss barely moved at all — from 0.1474 down to only 0.1453 across 10 epochs. BCE, on the other hand, kept improving the whole time. This happens because MSE's gradient shrinks a lot once the sigmoid output gets confident, which slows learning to a crawl (this is called a “vanishing gradient”). BCE doesn't have this problem — its gradient stays clean and simple. This test gives a real, measured reason for choosing BCE, instead of just quoting it from a textbook.

10.4 Final Model Evaluation

The trained model was tested on 1,418 images it had never seen before. Every number below was calculated by hand with NumPy — no outside library was used to score the model.

Metric	Value
Accuracy	90.34%
Precision	95.43%
Recall	93.14%
F1 Score	94.27%

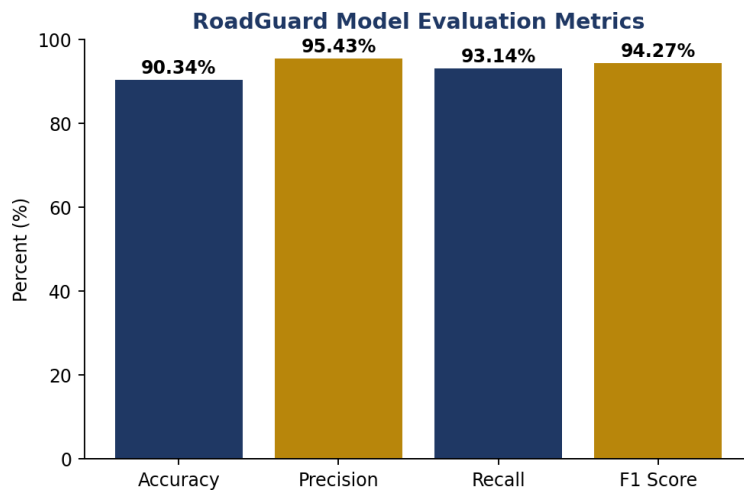


Figure 10.1 — The four evaluation metrics, from the real test run.

Confusion Matrix

	Predicted: Normal	Predicted: Damaged
Actual: Normal	154 (True Negative)	54 (False Positive)
Actual: Damaged	83 (False Negative)	1,127 (True Positive)

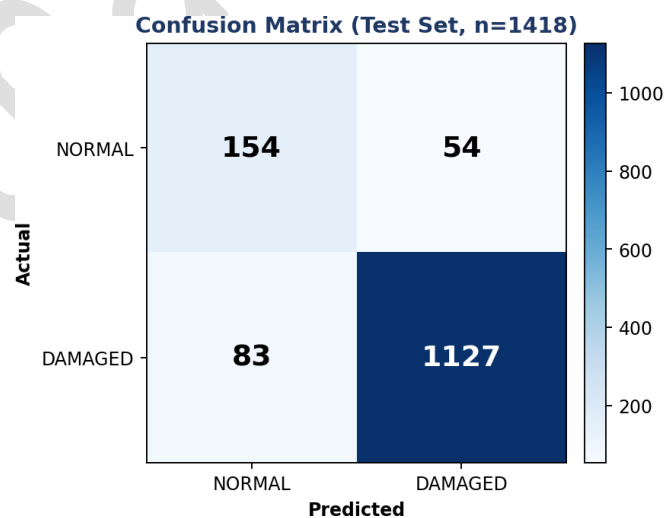


Figure 10.2 — Confusion matrix on the 1,418-image test set.

10.5 Discussion of Class Imbalance

Out of the 1,418 test images, 208 were really normal roads and 1,210 were really damaged roads — which shows the class imbalance mentioned in Chapter 5. The overall accuracy, precision, recall, and F1 numbers all look strong, but looking closer at the confusion matrix shows a real difference between the two classes:

- Recall on Damaged roads: $1,127 / 1,210 \approx 93.1\%$ — this matches the overall recall number, since damaged photos make up most of the test set.
- Recall on Normal roads: $154 / 208 \approx 74.0\%$ — clearly lower, meaning about 1 in 4 real normal roads got wrongly labeled as damaged.

This gap comes directly from the class imbalance (many more damaged photos than normal ones) — it's not a flaw in the network design or training. Overall accuracy can hide this kind of weakness when one class is much bigger than the other, which is why the confusion matrix and per-class recall are shown here instead of just relying on the top-line accuracy number.

Two practical fixes for this — adding more normal-road photos, and using class-weighted loss during training — are covered as future work in Chapter 11.

Chapter 11: Conclusion and Future Scope

11.1 Conclusion

RoadGuard successfully demonstrates that a neural network implemented entirely from scratch with NumPy — without any deep-learning framework — can achieve strong, measurable performance (90.34% accuracy, 94.27% F1 score) on a real, publicly sourced road-image classification task. The project integrates four core machine-learning practical topics (NumPy fundamentals, perceptrons and activation functions, forward/backpropagation, and loss functions/optimizers) into a single, working, end-to-end system that spans dataset preparation, model training, evaluation, a REST API, and a browser-based user interface supporting both file upload and live camera capture.

Throughout the project, particular care was taken to avoid fabricated data, labels, or results: the dataset-construction script refuses to invent a normal-road class when one does not genuinely exist in the source data, and the web application explicitly states when the model has not yet been trained rather than displaying placeholder metrics.

11.2 Limitations

- The dataset exhibits a significant class imbalance between damaged-road and normal-road images, which measurably reduces recall on the minority (normal) class, as shown in Chapter 10.
- The model operates on small, flattened 32×32 images rather than full-resolution photographs, limiting its ability to detect fine-grained or small-scale damage.
- As a fully connected network rather than a convolutional one, the model does not exploit spatial locality in the image, which may limit its accuracy ceiling compared to CNN-based approaches.
- The system classifies an entire image as Normal or Damaged; it does not localize the damaged region within the image.

11.3 Future Scope

- Class-weighted loss: Modify the Binary Cross-Entropy computation to weight the minority (normal) class more heavily during training, directly addressing the recall gap identified in Chapter 10.
- Additional normal-road images: Expand the normal-road dataset with more verified, real photographs to reduce the class-imbalance ratio.
- Manual data augmentation: Apply NumPy/OpenCV-based transformations (flips, brightness adjustment, minor rotation) to existing normal-road images to synthetically grow that class without fabricating new labels.
- From-scratch convolutional layers: Extend the NumPy implementation to include a manually coded convolutional layer, enabling the network to exploit spatial structure in the image.

- Damage localization: Incorporate a saliency or gradient-based visualization technique (leveraging the backpropagation gradients already computed) to highlight which regions of an image most influenced the model's decision.
- Deployment: Host the Flask API and React frontend on cloud platforms to make the system accessible beyond the local development environment

References

- [1] Kaggle — “Road Issues Detection Dataset,” available at: <https://www.kaggle.com/datasets/programmerrdai/road-issues-detection-dataset>
- [2] NumPy Documentation — <https://numpy.org/doc/>
- [3] OpenCV Documentation — <https://docs.opencv.org/>
- [4] Flask Documentation — <https://flask.palletsprojects.com/>
- [5] React Documentation — <https://react.dev/>
- [6] Vite Documentation — <https://vitejs.dev/>
- [7] Kingma, D. P., & Ba, J. — “Adam: A Method for Stochastic Optimization” (foundational reference for the Adam optimizer implemented manually in this project)

Appendix A: Project Structure

```
RoadGuard/  
|-- dataset/{train,validation,test}/{normal,damaged}/  
|-- ml_backend/  
|   |-- model/model.npz  
|   |-- results/*.png, metrics.json  
|   |-- prepare_dataset.py  
|   |-- practical_numpy.py  
|   |-- preprocessing.py  
|   |-- perceptron.py  
|   |-- activation_visualization.py  
|   |-- neural_network.py  
|   |-- data_loader.py  
|   |-- train.py  
|   |-- evaluation.py  
|   |-- predict.py  
|   |-- app.py  
|   |-- requirements.txt  
|-- src/ (React components, services, styles)  
|-- public/  
|-- package.json  
|-- vite.config.js
```